

Efficient Search Implementation

Milestone 3

Matt Krueger

May 29, 2025

The artifact, a Java project that tracks contacts and appointments using CRUD operations, was developed as part of my CS-320 software testing class. It includes a Contact class, a ContactService layer, and unit tests.

I selected this project after researching data structures used in my previous coursework. I found this project using a hash map for efficient lookups based on ID. However, this approach limits the usability of contacts since searching for contacts by their stored properties and while typing would be more convenient. To address this limitation, this enhancement is to implement a QuickSort or MergeSort algorithm with a time complexity of $O(\log n)$ to efficiently accomplish this search. This would demonstrate my mastery of algorithms and data structures.

I completed the implementation by implementing recursive partitioning, sorting, and searching functions. I wrote performance tests to verify that the execution time grew logarithmically. The implementation of partitioning, sorting, and searching was straightforward since these are well-documented algorithms. However, making them generic to any of my Java classes' properties posed a challenge. Initially, I had a significant amount of duplicated code that I struggled to rectify. Through research, I learned how to pass the different getter functions generically through function arguments. Initially, I overthought how to form my search query and decide which property to search on. To simplify the process, I wrote separate functions to search each property with minimal duplicated code. This allowed my query to consist of an exact search value or a prefix ending in a wildcard. I used test-driven development to ensure that the partitioning, sorting, and searching functions were correct.